Pasaporte BA

Arquitectura y modelado de datos

TP Integrador — Bases de Datos

Informe técnico final

Tecnicatura en Programación Informática — UNSAM

Docentes: Gabriel Flores y Rodrigo Casco

Integrantes: completar con los cinco nombres

Noviembre de 2026

MYSQL 8.4 | HBASE 2.1 | FASTAPI | EXPO 58

github.com/camitermine/Pasaporte-BA

1. Descripción del problema

Buenos Aires tiene miles de puntos de interés cultural, patrimonial, artístico y gastronómico publicados en fuentes separadas. Pasaporte BA los reúne en una aplicación móvil y web que permite explorar lugares por nombre, barrio, categoría o cercanía; guardarlos como favoritos; registrar visitas mediante check-ins; completar un pasaporte de barrios y consultar rankings.

Existen dos roles. El usuario común explora, marca favoritos, registra y administra sus propios check-ins y propone lugares. El administrador crea cuentas, mantiene el catálogo y aprueba o rechaza sugerencias. Los usuarios comunes no pueden agregar, modificar ni eliminar lugares de forma directa.

El problema combina dos clases de datos:

• Datos maestros y transaccionales: usuarios, sesiones, roles, lugares, categorías,

favoritos, sugerencias y registro canónico de check-ins. Requieren relaciones, restricciones y consistencia, por lo que MySQL es la fuente de verdad.

• Actividad y lecturas de alto volumen: historiales, pasaporte, rankings e índices de

consulta. Se benefician de filas anchas, acceso por clave y agregados precalculados, por lo que se proyectan en HBase.

2. Origen y descripción del dataset

Se integraron cuatro conjuntos del portal BA Data del Gobierno de la Ciudad de Buenos Aires. Las fichas oficiales indican licencia CC BY 2.5 AR. La fecha histórica exacta de descarga no quedó guardada junto con los CSV; para no inventarla, se registra que las fichas fueron consultadas el 12/11/2026.

Fuente local	Dataset oficial	Filas recibidas	URL oficial	Descarga / consulta	Licencia
`espacios_culturales`	Espacios culturales	3.056	https://data.buenosaires.gob.ar/dataset/espacios-culturales	descarga no registrada; consulta 12/11/2026	CC BY 2.5 AR
`monumentos`	Monumentos	2.051	https://data.buenosaires.gob.ar/dataset/monumentos	descarga no registrada; consulta 12/11/2026	CC BY 2.5 AR
`murales`	Murales	413	https://data.buenosaires.gob.ar/dataset/murales	descarga no registrada; consulta 12/11/2026	CC BY 2.5 AR
`oferta_gastronomica`	Oferta de establecimientos gastronómicos	2.812	https://data.buenosaires.gob.ar/dataset/oferta-establecimientos-gastronomicos	descarga no registrada; consulta 12/11/2026	CC BY 2.5 AR

El material recibido ya estaba integrado en cuatro archivos de trabajo dentro de `database/datasets/tablas_3fn/`: `lugares.csv`, `barrios.csv`, `tipos.csv` y `fuentes.csv`. El archivo principal tiene 8.332 filas y las columnas `id`, `nombre`, `fuente_id`, `tipo_id`, `calle`, `calle_interseccion`, `altura`, `barrio_id`, `comuna`, `latitud` y `longitud`.

Después de la limpieza se obtuvieron:

Entidad	Cantidad
Lugares válidos	8.322
Barrios	48
Comunas	15
Categorías de origen	228
Tipos agrupadores de la app	4
Fuentes	4

3. Análisis inicial y calidad de datos

El perfilado se realizó sobre los CSV conservados. Por fuente se observaron estos faltantes:

Fuente	Sin nombre	Sin barrio	Sin coordenadas completas
Espacios culturales	0	1	22
Monumentos	1	269	100
Murales	9	0	0
Oferta gastronómica	0	0	0
Total	10	270	122

También se encontraron repetición textual de barrios y categorías, dos representaciones de Núñez por codificación, categorías equivalentes por acentos, texto con mojibake, direcciones divididas en tres campos y un caso de coordenadas incompletas. Repetir barrio, comuna, categoría y tipo en cada lugar produciría anomalías de actualización e inconsistencias.

Las decisiones de limpieza fueron conservadoras: se descartaron solamente los diez registros sin nombre; los 270 lugares sin barrio se conservaron con FK nula; las coordenadas incompletas se convirtieron en nulas; se reparó texto cuando era inequívoco y se consolidaron duplicados de codificación. No se inventaron valores ausentes.

4. Evolución hasta tercera forma normal

La estructura plana común puede representarse así:

```
LUGAR_ORIGINAL(id_origen, nombre, fuente, categoria, tipo,
               calle, interseccion, altura, barrio, comuna,
               latitud, longitud)
```

Primera forma normal (1FN). Cada fila representa un lugar y cada celda un valor simple. Para el uso de la aplicación, `calle`, `interseccion` y `altura` se transforman en el atributo descriptivo `direccion`; no se almacenan listas ni grupos repetitivos.

Segunda forma normal (2FN). En cada fuente la clave original es simple. Al integrar fuentes, la identidad externa pasa a ser `(id_fuente, id_origen)`, combinación que queda protegida por una restricción `UNIQUE`. El nuevo `id_lugar` es la PK interna.

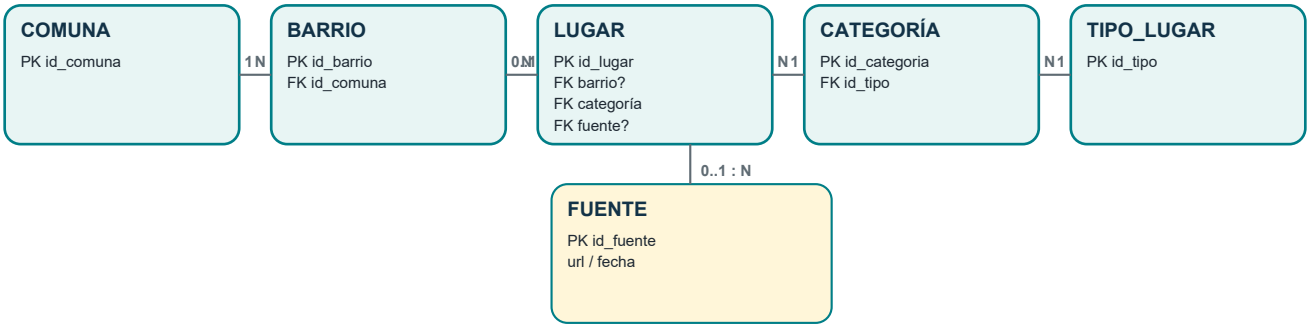
Tercera forma normal (3FN). Se separan las dependencias transitivas:

Dependencia	Descomposición aplicada
`lugar → barrio → comuna`	`COMUNA`, `BARRIO(id_comuna)` y `LUGAR(id_barrio)`
`lugar → categoría → tipo`	`TIPO_LUGAR`, `CATEGORIA(id_tipo)` y `LUGAR(id_categoria)`
`lugar → fuente → metadatos`	`FUENTE` y `LUGAR(id_fuente, id_origen)`

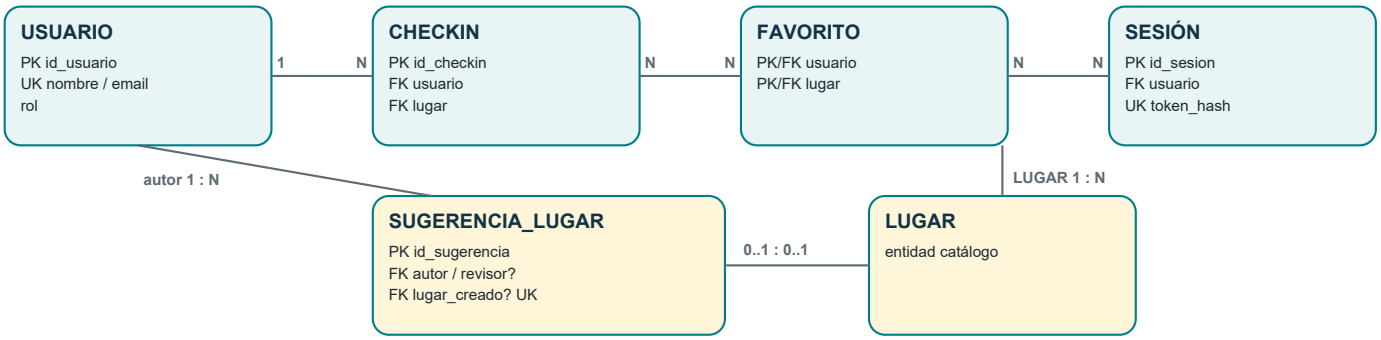
El resultado se completa con las entidades de la aplicación: `USUARIO`, `CHECKIN`, `FAVORITO`, `SUGERENCIA_LUGAR` y `SESION`. El ETL reproducible `database/etl/generar_datos_mysql.py` lee los CSV sin modificarlos y genera `database/mysql/datos_iniciales.sql`.

5. Diagrama Entidad–Relación

DER — catálogo normalizado



DER — actividad, seguridad y moderación



CHECKIN y FAVORITO resuelven las relaciones N:M entre USUARIO y LUGAR.

Figura 1. DER resumido. El detalle de atributos y cardinalidades se conserva en `documentacion/DER.md`.

El DER completo y la explicación de cardinalidades están en `documentacion/DER.md`. Sus relaciones principales son:

```
COMUNA 1 ── N BARRIO 0..1 ── N LUGAR N ── 1 CATEGORIA N ── 1 TIPO_LUGAR
                                  │
FUENTE 0..1 ──────────────── N ┤
                                  ├── N CHECKIN N ── 1 USUARIO ── N SESION
                                  ├── N FAVORITO N ─ 1 USUARIO
                                  └── 0..1 SUGERENCIA_LUGAR N ─── 1 USUARIO (autor)
```

`CHECKIN` y `FAVORITO` resuelven relaciones N:M entre usuario y lugar. Una sugerencia aprobada puede crear como máximo un lugar, y un lugar puede estar asociado como máximo a una sugerencia; esto se garantiza con una FK nullable y `UNIQUE (id_lugar_creado)`.

6. Modelo relacional

Se implementó MySQL 8.4 en `database/mysql/modelo_relacional.sql`.

Tabla	Clave y restricciones destacadas
`comuna`	PK `id_comuna`; `CHECK` entre 1 y 15
`barrio`	PK; FK a comuna; nombre único
`tipo_lugar`	PK; nombre único
`categoria`	PK; FK a tipo; `(nombre, id_tipo)` único
`fuente`	PK; nombre único; URL y fecha de descarga nullable
`lugar`	PK; FK a categoría, barrio y fuente; origen único; coordenadas; baja lógica
`usuario`	PK; usuario y email únicos; rol; contraseña hasheada; baja lógica
`checkin`	PK; FK a usuario y lugar; puntaje 1–5; comentario; verificación

Tabla	Clave y restricciones destacadas
`favorito`	PK compuesta `(id_usuario, id_lugar)`
`sugerencia_lugar`	PK; autor, revisor, estado y lugar creado único nullable
`sesion`	PK; hash de token único; vencimiento y revocación

Las FKs protegen la integridad. Los lugares usan baja lógica porque un lugar visitado no debe perder su historial. Las contraseñas se guardan con PBKDF2-SHA256 y sal aleatoria; de cada sesión solo se persiste el hash SHA-256 del token. Los índices compuestos siguen los accesos reales por usuario/lugar y fecha.

7. Diseño NoSQL

Se eligió Apache HBase 2.1, base orientada a columnas anchas. Su utilización fue autorizada por los docentes y permite modelar directamente los patrones de lectura de actividad. HBase no reemplaza el modelo relacional: contiene una proyección reconstruible y optimizada.

Tabla HBase	Row key	Acceso que resuelve
`lugares`	`<idLugar>`	detalle desnormalizado
`lugares_por_barrio`	`<idBarrio>#<idLugar>`	lugares de un barrio
`lugares_por_categoria`	`<idCategoria>#<idLugar>`	lugares de una categoría
`lugares_por_geohash`	`<geohash6>#<idLugar>`	candidatos cercanos
`usuarios`	`<idUsuario>`	perfil necesario para actividad
`usuarios_por_nombre`	`<nombreUsuario>`	índice de búsqueda
`checkins`	`<idCheckin>`	detalle de check-in
`checkins_por_lugar`	`<idLugar>#<tsInvertido>#<idCheckin>`	actividad reciente de un lugar
`checkins_por_usuario`	`<idUsuario>#<tsInvertido>#<idCheckin>`	historial y pasaporte
`stats_lugar`	`<idLugar>`; columnas `m:yyyyMM` y `t:total`	ranking y totales
`secuencias`	nombre de entidad	contadores de IDs

Los IDs se rellenan con ceros para preservar orden lexicográfico; el timestamp invertido ubica primero lo más reciente; el geohash convierte cercanía en scans por prefijo; y los contadores atómicos evitan agrupar millones de check-ins en cada ranking. Un mismo check-in se escribe en tres tablas porque cada tabla responde un acceso concreto.

Los lugares sin barrio también se proyectan en `lugares` y en los índices de categoría/geohash; simplemente no generan una fila en `lugares_por_barrio`.

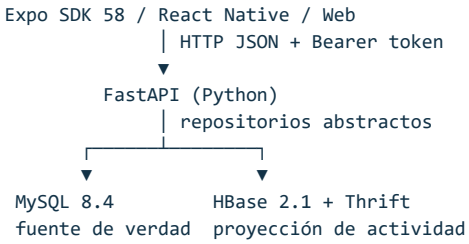
8. Justificación de HBase, ventajas y limitaciones

La parte beneficiada por NoSQL es la actividad: check-ins, historiales por usuario o lugar, pasaporte y rankings. Es un flujo creciente, mayormente append, con claves de acceso conocidas. HBase permite escalar horizontalmente y hacer scans de rangos contiguos sin joins.

Sus ventajas son las lecturas preordenadas por row key, contadores atómicos, esquema flexible y rendimiento estable de agregados precalculados. Sus costos son la duplicación, ausencia de FKs, `UNIQUE`, joins y transacciones entre filas, mayor complejidad operativa y necesidad de diseñar una tabla por patrón de consulta. La aplicación debe validar integridad y coordinar escrituras.

Por eso MySQL conserva la verdad canónica. Una escritura de actividad se realiza primero en MySQL y luego se refleja en HBase. La proyección completa puede regenerarse con `database/hbase/cargar_desde_mysql.py`.

9. Arquitectura general



La capa de presentación nunca accede a las bases directamente. FastAPI expone la API y usa interfaces comunes de repositorios. Docker Compose levanta MySQL, HBase, inicialización de HBase, backend y frontend web.

10. Backend polimórfico y funcionalidades

`backend/app/persistencia/base.py` define las interfaces `RepoLugares`, `RepoCheckins`, `RepoConsultas`, `RepoUsuarios` y `RepoCatalogo`. MySQL y HBase implementan esas operaciones con SQL o con row keys/scans. La API depende de las interfaces y la fábrica decide la implementación.

En modo automático, MySQL resuelve cuentas, catálogo y transacciones; HBase resuelve historiales, pasaporte y rankings. En modo demostración, el header `X-Motor: mysql|hbase` permite ejecutar las lecturas que cuentan con ambas implementaciones y comparar sus metadatos. No es una elección funcional del usuario: la aplicación decide la base apropiada. Autenticación, favoritos y sugerencias son exclusivamente relacionales y no obedecen al selector. Las escrituras siempre son canónicas en MySQL y, cuando corresponde, se proyectan a HBase.

Se implementó CRUD de lugares para administradores y CRUD de check-ins para su autor o un administrador. También hay login/logout, roles, favoritos, sugerencias moderadas, búsqueda y filtros, cercanía, pasaporte y ranking mensual. Cada respuesta informa motor, operaciones y duración en `meta`, visibles en la barra de diagnóstico de la app.

11. Tecnologías y organización

Capa	Tecnología
Base relacional	MySQL 8.4
Base NoSQL	Apache HBase 2.1, Thrift y HappyBase
Backend	Python 3.11+, FastAPI, SQLAlchemy, PyMySQL y Pydantic
Frontend	Expo SDK 58, React Native, Expo Router y TypeScript
Infraestructura	Docker Compose y nginx para la versión web
Versionado	Git y GitHub, ramas por funcionalidad y merges a `main`
Gestión	Backlog y decisiones en README/documentación; coordinación por ramas y commits

No se usó una herramienta externa de gestión de tareas. Esta decisión simplificó un grupo y un alcance pequeños, aunque para una segunda etapa convendría usar GitHub Projects con responsables y criterios de aceptación.

12. Ejecución reproducible

La forma recomendada requiere Git y Docker Desktop. Desde la raíz:

```
docker compose up -d --build
docker compose ps
```

La aplicación web queda en http://localhost:8081, la documentación de la API en http://localhost:8000/docs y la UI de HBase en http://localhost:16010. MySQL crea esquema y datos iniciales al crear el volumen; `hbase-init` reconstruye la proyección antes de iniciar el backend. Las instrucciones de desarrollo con Python, Node y Expo Go

están detalladas en `README.md`.

13. Ejemplos de funcionamiento

1. El usuario inicia sesión y recibe un token revocable.
2. Busca “museo”, filtra por barrio/categoría y abre el detalle de un lugar.
3. Guarda el lugar como favorito y registra un check-in con puntaje y comentario. Como todavía no se adjunta foto, queda `verificado = false`.
4. El pasaporte consulta barrios visitados y calcula el progreso; el ranking muestra los lugares más visitados de un barrio durante un mes.
5. Un usuario propone un lugar. Un administrador lo aprueba; recién entonces se crea el lugar y se proyecta a HBase.
6. Un administrador crea, modifica o da de baja un lugar. El usuario común recibe un error 403.
7. En Ajustes se fuerza MySQL o HBase para repetir una lectura comparable; la app muestra el motor y los tiempos devueltos por la API.

La API interactiva permite reproducir estos flujos y ver los esquemas de entrada/salida en `/docs`. Las cuentas locales de demostración son `admin / Admin123!` y `visitante / Usuario123!`.

14. Comparación MySQL–HBase

Mediana con 1.000.001 check-ins (ms; menor es mejor)

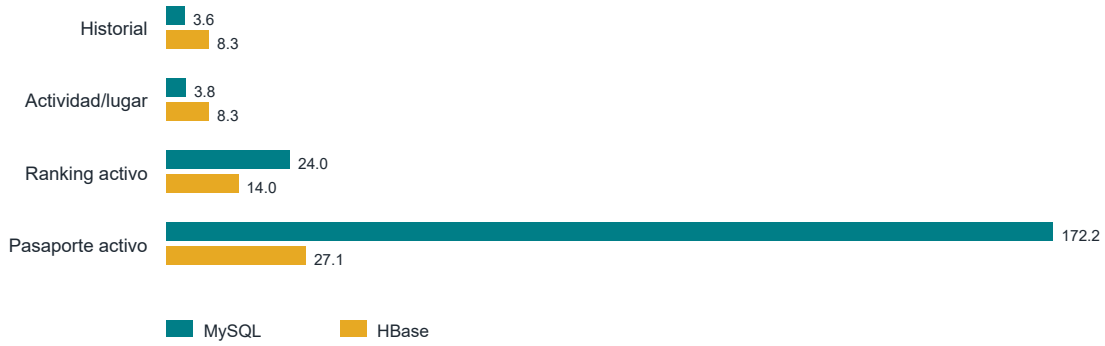


Figura 2. El costo fijo de HBase penaliza lecturas pequeñas; sus agregados resultan ventajosos al crecer el volumen.

`backend/dev/comparar_motores.py` ejecuta 30 repeticiones, compara resultados y registra mediana, p95 y mínimo. La actividad sintética es reproducible con semilla fija. En el escenario de 1.000.001 check-ins, todas las filas reportaron `equivalentes=True`.

Consulta	Caso	MySQL mediana (ms)	HBase mediana (ms)
Historial por usuario	más activo, 1.303 check-ins	3,6	8,3
Actividad por lugar	más activo, 59.899 check-ins	3,8	8,3
Ranking mensual	barrio más activo, 13.463 check-ins	24,0	14,0
Ranking mensual	barrio mediano	6,9	9,1
Pasaporte	usuario más activo	172,2	27,1
Pasaporte	usuario mediano	9,9	10,0

Los resultados completos para aproximadamente 10.000, 100.000 y 1.000.000 de check-ins están en `backend/dev/resultados/benchmark.csv`. MySQL gana en lecturas puntuales y paginadas gracias a sus índices

y al menor costo de conexión. HBase gana cuando el volumen hace costosos los agrupamientos y puede aprovechar contadores o scans preordenados. No existe un ganador universal: depende del patrón de acceso y del volumen.

15. Problemas encontrados y decisiones

Problema	Decisión aplicada
Núñez con dos codificaciones	consolidación en el ETL
Categorías equivalentes por acentos	comparación normalizada compatible con la collation
Diez lugares sin nombre	descarte explícito, sin inventar nombres
270 lugares sin barrio	FK nula; HBase los conserva fuera del índice por barrio
Coordenadas incompletas	par de coordenadas nulo
HBase sin integridad relacional	validación de aplicación y MySQL como fuente canónica
Duplicación NoSQL	proyección reconstruible desde MySQL
Riesgo de desincronización	escritura MySQL primero y proyección inmediata; recarga completa disponible
Comparar motores sin duplicar API	interfaces comunes, selector solo para lecturas equivalentes y metadatos
Mapas nativos no disponibles en web	componente por plataforma y OpenStreetMap en web
Nombres de origen en mayúsculas o sin tildes	presentación en formato título; se preserva el dato trazable
Falta de fecha histórica de descarga	se declara la ausencia y la fecha de consulta, sin inventar metadatos

16. Conclusiones

La normalización permitió convertir cuatro fuentes heterogéneas en un modelo coherente y evitar anomalías reales de actualización. Las restricciones de MySQL resultan esenciales para usuarios, roles, moderación y trazabilidad. HBase no se eligió para reemplazarlo, sino para una parte concreta que crece y tiene accesos repetitivos por usuario, lugar y mes.

La comparación confirma que la tecnología adecuada depende del caso: MySQL es superior para transacciones y lecturas pequeñas; HBase muestra ventaja en agregaciones de gran volumen preparadas por su modelo. Separar la API de la persistencia permitió implementar y medir ambos motores sin duplicar la lógica funcional.

La principal contrapartida de NoSQL es trasladar al código responsabilidades que el motor relacional resuelve con FKs y transacciones. Como evolución quedan la verificación del check-in mediante foto y ubicación, amistades/feed, insignias, rutas y despliegue público; esas funciones no forman parte de la entrega actual y no alteran el modelo base.

17. Correspondencia con las consignas

Consigna	Evidencia en el proyecto
Problema y alcance	secciones 1 y 10; frontend y API funcionales
Dataset, origen y análisis	secciones 2 y 3; `database/datasets/`
Normalización hasta 3FN	sección 4; ETL reproducible
DER y cardinalidades	sección 5; `documentacion/DER.md`
Modelo relacional, claves y restricciones	sección 6; `modelo_relacional.sql`
CRUD	lugares y check-ins en sección 10 y endpoints FastAPI
Consultas no triviales	cercanos, pasaporte y ranking mensual
Parte beneficiada por NoSQL	secciones 7 y 8; actividad e índices HBase

Consigna	Evidencia en el proyecto
Ventajas y limitaciones NoSQL	sección 8
Dos implementaciones comparables	interfaces de persistencia y `X-Motor`
Comparación experimental	sección 14; script y CSV de resultados
Aplicación, arquitectura y tecnologías	secciones 9 a 13; Docker Compose
Problemas y conclusiones	secciones 15 y 16

Referencias

- Gobierno de la Ciudad de Buenos Aires, portal BA Data: https://data.buenosaires.gob.ar.
- Apache HBase Reference Guide: https://hbase.apache.org/book.html.
- MySQL 8.4 Reference Manual: https://dev.mysql.com/doc/refman/8.4/en/.
- FastAPI Documentation: https://fastapi.tiangolo.com/.